OASIS Committee Note

XLIFF Translation Memory Version 1.0

Committee Note Draft 01

8 July 2026

Specification URIs

This version:
[link to authoritative version of the published document] (Authoritative)

Previous version:
[link to authoritative version of the published document] (Authoritative)

Latest version:
[link to authoritative version of the published document] (Authoritative)

Technical Committee:
OASIS XLIFF TC

Chairs:
Dr. Lucía Morado Vázquez (lucia.morado@unige.ch), University of Geneva
Yoshito Umaoka (y.umaoka@gmail.com), Individual

Secretary:
Rodolfo M. Raya (rmraya@maxprograms.com), Maxprograms

Editor:
Rodolfo M. Raya (rmraya@maxprograms.com), Maxprograms

Related Work:
This document replaces or supersedes:

- [The full reference to the related document in IEEE reference format]

This document is related to:

- **[XLIFF-2.3-part2]** XLIFF Version 2.3 Part 2: Extended. Edited by Rodolfo M. Raya and Lucía Morado Vázquez 8 July 2026. OASIS Committee Specification Draft 01. https://docs.oasis-open.org/xliff/xliff-core/v2.3/xliff-v2.3-part2-extended.html.
- **[TMX 1.4b]** Translation Memory eXchange (TMX) Version 1.4b. Edited by Yves Savourel. OSCAR Recommendation, 26 April 2005. https://web.archive.org/web/20060528180452/http://www.lisa.org/standards/tmx/tmx.html.

Abstract:
This Committee Note describes an XLIFF-native approach for the representation and exchange of translation memory data, using plain XLIFF `<file>`, `<unit>`, and `<segment>` elements as the translation memory store itself, in place of the Translation Memory eXchange (TMX) format. The approach preserves the structure and semantics of XLIFF content, including bilingual text, inline elements, segmentation, and associated metadata, without the information loss that results from

converting content to and from TMX's inline tag model. It also preserves the document order and structural relationship between translation units, file, unit, and segment, that TMX's flat, unordered list of translation units does not represent, and it allows richer metadata, such as provenance information, to be attached directly using existing XLIFF modules.

This Committee Note also addresses the exchange of translation memory data for multilingual use. Because XLIFF documents are fundamentally bilingual, the Note recommends a packaging approach in which multiple related bilingual XLIFF files are distributed together in a single ZIP package with a manifest that identifies the relationships among them.

Citation format:

When referencing this document, the following citation format should be used:

[The full reference for this document in IEEE reference format]

License, Document Status, and Notices

Table of Contents

Appendixes

1 Scope

This Committee Note describes an XLIFF-native approach for storing and exchanging translation memory (TM) data, using plain XLIFF 2.x documents in place of the Translation Memory eXchange (TMX) format.

Rather than defining a new container or converting bilingual content into a separate TM-specific format, this document recommends that XLIFF `<file>`, `<unit>`, and `<segment>` elements be used directly to hold translation units for later reuse. This preserves the full XLIFF inline element model, avoiding the loss of information that results from converting XLIFF inline markup to and from the TMX inline tag model, and preserves the structural order of content, which TMX's flat, unordered list of `<tu>` elements does not represent.

This document also describes how the XLIFF Provenance module, under development for XLIFF 2.3, can be used to record and later query information about the origin of stored translation units, such as the agent, tool, or process that produced or modified them.

Because a single XLIFF document is inherently bilingual, this document further describes a packaging convention for exchanging translation memory data across more than two languages, based on a set of related bilingual XLIFF files distributed together with a manifest.

2 Definitions and Acronyms

2.1 Definitions

2.1.1 Terms Defined Elsewhere

This document uses the following terms defined elsewhere:

Unit [XLIFF-2.3-part2]
: "A `<unit>` MAY contain both source and target content, allowing bilingual data to be stored."

Segment [XLIFF-2.3-part2]
: The `<segment>` element, used to represent the smallest unit of translatable content addressed for translation, review, or other processing.

Provenance information [Provenance module, XLIFF 2.3, informative]
: Information about the nature and origin of the localization data or process.

2.1.2 Terms Defined in this Document

This document defines the following terms:

Translation memory (TM)
: A collection of previously translated source and target content, organized for retrieval and reuse when equal or similar source content is encountered again.

Translation memory entry
: A single stored source/target pair available for reuse, represented in this document as an XLIFF `<unit>` (or, where segmented, `<segment>`).

Translation memory exchange package
: A ZIP archive containing one or more bilingual XLIFF files together with a manifest, used to exchange translation memory data across more than two languages, as described in Exchange of Multilingual Translation Memory Data.

2.2 Abbreviations and Acronyms

This document uses the following abbreviations and acronyms:

- TC: Technical Committee
- TM: Translation Memory
- TMX: Translation Memory eXchange
- TU: Translation Unit
- TUV: Translation Unit Variant (TMX)
- XML: Extensible Markup Language
- ZIP: A common archive file format

3 Document Conventions

3.1 Key Words

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

3.2 Typographical Conventions

XML element names are shown enclosed in angle brackets, for example `<unit>`. XML attribute names are shown in fixed-width type without brackets, for example `id`. XML namespace prefixes used in examples (`xlf:`, `mtc:`, `pvn:`) follow the conventions established in [XLIFF-2.3-part2] and the XLIFF 2.3 Provenance module. Example XML is shown in fenced code blocks and is illustrative; it is not necessarily complete or schema-valid.

4 Introduction

Translation memory (TM) is a core technology of the localization industry. A TM stores previously translated content so that it can be retrieved and reused when equal or similar source content is encountered again, improving consistency and reducing translation effort and cost.

Since 2005, the de facto standard for exchanging TM data between tools has been the Translation Memory eXchange (TMX) format [TMX 1.4b]. TMX was designed as a vendor-neutral interchange format, independent of any single authoring or extraction format. XLIFF, by contrast, is designed to carry bilingual (and, through packaging, multilingual) content through the localization process itself, from extraction through translation, review, and reintegration into the original format.

As XLIFF adoption has grown, and as the XLIFF inline content model, segmentation model, and metadata modules have become richer, the gap between what an XLIFF document can represent and what TMX can represent has widened. Producing a TMX export of translated content, or importing TMX leverage into an XLIFF-based workflow, requires a conversion step. That conversion step is where information is put at risk.

4.1 Limitations of TMX-Based Exchange

TMX and XLIFF do not share an inline content model. TMX represents inline markup with a small, generic set of placeholder elements (`<bpt>`, `<ept>`, `<it>`, `<ph>`, `<hi>`) that carry no structural relationship to the original markup beyond matched pairing and free-form type/i attributes. XLIFF represents inline content with a richer, self-describing model (`<pc>`, `<sc>`, `<ec>`, `<ph>`, `<mrk>`, `<sm>`/`<em>`, backed by `<originalData>`). Converting XLIFF inline content into TMX inline content, and back again, is a lossy, implementation-specific mapping. There is no guarantee that a round trip through TMX preserves the original XLIFF markup, and in practice it frequently does not.

TMX is defined by a Document Type Definition (DTD), not an XML Schema, and a TMX document is a self-contained top-level document with its own DOCTYPE. A TMX `<tu>` cannot be embedded inside an XLIFF document, and an XLIFF `<unit>` cannot be embedded inside a TMX document. The two formats can only be related to each other through external conversion, never through direct inclusion. This also means TMX content cannot participate in XLIFF's own extension and modularization mechanisms, such as the Translation Candidates module (`mtc:matches`, see [XLIFF-2.3-part2]), which is designed to hold XLIFF-native `<source>`/`<target>` pairs as leveraging matches inside a unit.

Finally, a TMX file is a flat, unordered collection of `<tu>` elements. TMX defines no relationship between one translation unit and the next; any notion of document order, or of which segments originally appeared near each other, is lost once content is exported to TMX (short of ad hoc use of `<prop>` or `<note>` elements to reconstruct it). XLIFF does not have this limitation: an XLIFF document is structured as `<file>` containing one or more `<unit>` elements, each containing one or more `<segment>` elements, and that hierarchy is preserved in document order. Two segments that are adjacent in an XLIFF file are known to be adjacent in the source content, which is valuable context for TM leveraging, concordance search, and quality review that a flat TMX export cannot provide.

For these reasons, this document recommends that plain XLIFF documents be used directly as the storage and interchange format for translation memory data, rather than converting content to and from TMX.

4.2 Changes From the Previous Version

The list of changes from the previous version and any revision history can be found in Appendix 2.

5 XLIFF as a Translation Memory Format

This section describes the recommended approach for storing translation memory data as plain XLIFF documents, using XLIFF Core elements as they are already defined, without introducing a new TM-specific vocabulary.

5.1 Design Principles

- **No conversion.** A TM entry is an XLIFF `<unit>` containing one or more `<segment>` elements with `<source>` and `<target>` content, exactly as produced by any XLIFF-conformant extraction, translation, or review tool. Storing that content as translation memory requires no transformation of inline markup, and retrieving it for reuse in another XLIFF document requires no transformation back.
- **Full inline fidelity.** Because TM entries remain XLIFF `<unit>` elements, they retain the full XLIFF inline element model (`<pc>`, `<sc>`/`<ec>`, `<ph>`, `<mrk>`, `<sm>`/`<em>`) together with any associated `<originalData>`, instead of being reduced to the smaller TMX inline tag set.
- **Preserved order.** Units and segments remain in the document order in which they were extracted, inside their originating `<file>` (and, where used, `<group>`). See Structural Order and Segment Context.
- **Extensibility through existing modules.** Any XLIFF module that can annotate a `<file>`, `<unit>`, or `<segment>` in a normal XLIFF document, such as metadata, notes, size and length restriction, or the Provenance module described in Provenance Metadata for Translation Memory Entries, applies equally when that document is used as a TM store. No parallel metadata model is required.

5.2 Harvesting Translation Memory Entries

A translation memory store, as described in this document, is not authored directly. It is populated by harvesting completed translation work out of ordinary XLIFF documents produced by an extraction, translation, and review process, and copying the resulting `<unit>` and `<segment>` elements, unchanged, into the store.

Only `<segment>` elements whose `state` attribute [XLIFF-2.3-part2] is set to `final` SHOULD be harvested into a translation memory store by default. The `initial`, `translated`, and `reviewed` values of `state` represent work still in progress; storing a segment in one of those states as reusable translation memory risks propagating unreviewed or incomplete translations into unrelated future projects. A segment that carries no `state` attribute, which defaults to `initial`, is likewise not eligible for harvesting. An implementation MAY instead apply a configurable minimum state threshold, for example accepting both `reviewed` and `final` segments, provided the threshold it used is disclosed to whoever queries the resulting store.

Because the same project file may be harvested more than once, for example after a partial re-export, an implementation SHOULD derive a stable identifier for each harvested entry from values that do not change between harvests, such as the enclosing `<file>` `id` (or `original` attribute), the `<unit>` `id`, and the segment's position within the unit. Deriving entry identifiers this way keeps harvesting idempotent: re-harvesting an unchanged file does not create duplicate entries in the store.

Because harvesting is a filter rather than a transformation, it does not conflict with the "No conversion" design principle in Design Principles: a harvested entry keeps exactly the source, target, and inline content it had in the file it was extracted from. Metadata recording where and how an entry was harvested, such as the originating project or job and the date of harvest, is expressed using the Provenance module, described in Provenance Metadata for Translation Memory Entries, rather than by any new attribute defined by this document.

5.3 Structural Order and Segment Context

TMX represents a translation memory as a flat, unordered collection of <tu> elements. TMX defines no relationship between one translation unit and any other; the position of a <tu> in a TMX file carries no meaning.

An XLIFF document used as a TM store retains the <file> > <unit> > hierarchy defined by XLIFF Core. This hierarchy carries information that a flat TU list cannot:

- **Document order** is preserved. Segments that were adjacent in the original content remain adjacent, which supports context-aware leveraging (for example, preferring a match whose neighboring segments also match) and concordance search that shows a hit in its original surrounding context.
- **Provenance of the source document** is preserved. The enclosing <file> retains attributes such as original, and can carry <file>-level metadata, so that every stored <unit> remains traceable to the document it came from.
- **Grouping** is preserved. Where the source content used <group> to represent structural divisions (chapters, UI screens, table rows, and so on), those divisions remain available to a TM consumer, instead of being discarded at export time.

Combining the last two properties lets a TM consumer recover an in-context exact match: a 100% text match whose neighboring segments and originating document, identified by the enclosing <file> element's original attribute, also match the segment currently being translated. Tools commonly treat such a match as more reliable than an exact match recovered from an unrelated document or context, and can apply it without requiring human review. A flat TMX export, lacking both an equivalent to original and any notion of adjacency, cannot make this distinction without falling back to ad hoc <prop> conventions.

A translation memory consumer that only needs unordered TU-style lookup can still treat every in every <unit> of every <file> as an independent entry, exactly as it would treat entries in a TMX file. The XLIFF-based representation is a strict superset of that capability: it does not require order and context to be used, but it does not discard them either.

5.4 Provenance Metadata for Translation Memory Entries

Reuse decisions in a translation memory workflow are often qualified by information about the origin of a given entry: who or what produced or last modified it, when, using what tool, and why. TMX addresses this only partially, through the creationid, creationdate, changeid, changedate, and <prop>/<note> mechanisms defined for <tu> and <tuv>.

The XLIFF 2.3 Provenance module (pvn:, under development by the OASIS XLIFF TC, see the xliff-23 provenance module in this repository) is designed to record equivalent, and richer, information directly on XLIFF Core elements, using the <pvn:provenance> and <pvn:change> elements and attributes such as agent, person, organization, tool, timestamp, appliesTo, and intent. Because a TM entry stored as described in this document is an ordinary XLIFF <unit>, it can carry <pvn:provenance> metadata exactly as any other XLIFF unit can.

Provenance is particularly important for entries populated through the harvesting process described in Harvesting Translation Memory Entries. Because a store accumulates entries across many separate harvest events, drawn from unrelated projects and completed at different times, the <pvn:provenance> and <pvn:change> elements are the mechanism by which a stored entry remains traceable back to the project, job, or process that produced it and the date it was harvested, once it is no longer possible to infer that origin from the entry's position in the store alone.

This gives a translation memory consumer the ability to query stored entries by their provenance, for example to prefer entries produced by a specific translator or reviewed by a specific organization, to exclude machine-translated entries that were never reviewed by a person, or to filter by the date a translation was performed. Since provenance metadata is expressed using XLIFF elements and attributes rather than free-text `<prop>` values, as TMX requires, it can be validated and queried using standard XML tooling.

5.5 Example

A single translated document only ever has one `<file>`, so nothing about it shows translation memory reuse. The following example instead shows a TM store: an XLIFF document aggregating entries extracted from two unrelated projects, translated five months apart. The second project reuses a sentence already translated for the first, which is exactly the situation a translation memory exists to detect and exploit. Because both projects remain inside a single XLIFF document, each entry stays traceable to the `<file>` (and therefore the project) it came from, in the order it was added to the store, which a flat TMX `<tu>` list cannot represent.

```
<xliff version="2.3" srcLang="en" trgLang="es"
       xmlns="urn:oasis:names:tc:xliff:document:2.3"
       xmlns:pvn="urn:oasis:names:tc:xliff:pvn:2.3">
  <file id="f1" original="manual/chapter3.html">
    <unit id="tm-0001">
      <segment>
        <source>Your annual tax certificate is now available for download.</source>
        <target>Su certificado fiscal anual ya está disponible para descarga.</target>
      </segment>
    </unit>
    <unit id="tm-0002">
      <pvn:provenance>
        <pvn:change agent="Acme MT" appliesTo="target" timestamp="2026-01-15T10:15:00.0
      </pvn:provenance>
      <segment>
        <source>Download your certificate before the end of the fiscal year.</source>
        <target>Descargue su certificado antes de que finalice el año fiscal.</target>
      </segment>
    </unit>
  </file>
  <file id="f2" original="app/ui-strings.json">
    <unit id="tm-1001">
      <pvn:provenance>
        <pvn:change agent="TM Reuse" appliesTo="target" timestamp="2026-06-30T09:02:00.
          intent="100% match reused from tm-0002 (manual/chapter3.html)."/>
      </pvn:provenance>
      <segment>
        <source>Download your certificate before the end of the fiscal year.</source>
        <target>Descargue su certificado antes de que finalice el año fiscal.</target>
      </segment>
    </unit>
    <unit id="tm-1002">
      <segment>
        <source>Your session will expire in five minutes.</source>
        <target>Su sesión caducará en cinco minutos.</target>
      </segment>
    </unit>
  </file>
</xliff>
```

Because `tm-1001` and `tm-0002` share the same `<source>` text, a TM consumer can find the reuse with a plain text comparison across the store, exactly as it would across a flat TMX file, then attribute that reuse using the Provenance module's `intent` attribute instead of a free-text TMX `<prop>`. And because both entries remain inside `<file>` elements that retain their own `original` attribute, the store also records which project each translation came from and in what order it was added, something a TMX export of the same two projects would have discarded.

6 Exchange of Multilingual Translation Memory Data

An XLIFF document is bilingual: it declares a single `srcLang` and a single `trgLang`. A translation memory, however, is often required to hold or exchange content across more than two languages. TMX addresses this within a single file, because a `<tu>` can hold any number of `<tuv>` elements, one per language.

This document recommends addressing the same requirement not by making an individual XLIFF document multilingual, which would be a departure from the XLIFF Core data model, but by packaging multiple bilingual XLIFF files together, one per source/target language pair, along with a manifest that identifies the relationship among them. This keeps every individual file fully conformant with, and processable by, any existing XLIFF 2.x tool, while still allowing a translation memory covering many languages to be exchanged as a single unit.

A multilingual translation memory is rarely built from a single source language: it typically accumulates from projects extracted from whatever source language each originating document happened to use. Accordingly, the files in a package are not required to share a common `srcLang`. A single package MAY combine bilingual files representing any number of independent source/target language pairs, for example an `en-es.xlf` file alongside an unrelated `fr-de.xlf` file, with the manifest simply recording the `srcLang` and `trgLang` that each file declares.

6.1 Package Structure

A translation memory exchange package is a ZIP archive (as defined in [ZIP APPNOTE]) with the following content:

- One or more bilingual XLIFF files, each conforming to [XLIFF-2.3-part2] or later, each independently declaring its own `srcLang` and `trgLang`. Files in the same package are not required to share a common `srcLang`.
- A single manifest file, described in Manifest File, that lists the XLIFF files in the package and the language pair each one represents.

The XLIFF files in a package are not required to sit at the root of the archive. They MAY instead be organized into any folder structure the package author finds convenient, for example one folder per language pair. The manifest's `href` value for each file gives that file's actual path within the package, so a consumer locates every file through the manifest rather than by assuming a particular layout.

Where two or more files in a package share the same `srcLang` and were extracted from the same source content, corresponding translation units across those files SHOULD use the same `<unit>` `id` value (and, where segmented, the same `<segment>` `id` value), so that a consumer can align entries for the same source content across target languages. This mirrors, at the package level, the alignment that a multilingual `<tu>` provides in a single TMX file for that case. This alignment applies only among files that share a source: it does not extend across files with different `srcLang` values, whose `<unit>` and `<segment>` `id` values are independent of one another.

6.2 Manifest File

The manifest is an XML file, named `manifest.xml`, placed at the root of the package. It lists each XLIFF file contained in the package, together with its source and target language, using an `href` value that gives the file's path within the package.

```
<tmPackage xmlns="urn:oasis:names:tc:xliff:tm-manifest:1.0" version="1.0">
  <file href="en-es.xlf" srcLang="en" trgLang="es"/>
```

```
  <file href="en-fr.xlf" srcLang="en" trgLang="fr"/>
  <file href="fr-de.xlf" srcLang="fr" trgLang="de"/>
</tmPackage>
```

A manifest is not required to declare the same `srcLang` for every file it lists, as the third entry above illustrates: `en-es.xlf` and `en-fr.xlf` both originate from English source content, while `fr-de.xlf` is an unrelated bilingual file with French as its source language.

The manifest namespace, element and attribute names, and XML Schema shown above are provisional and subject to review by the OASIS XLIFF TC before this Note advances beyond Committee Note Draft.

6.3 Example

A translation memory accumulated from an English-source project translated into Spanish and French, together with an unrelated project whose source was French and whose target was German, would be packaged as:

```
tm-package.zip
    manifest.xml
    en-es.xlf
    en-fr.xlf
    fr-de.xlf
```

Each of `en-es.xlf`, `en-fr.xlf`, and `fr-de.xlf` is an ordinary bilingual XLIFF document as described in XLIFF as a Translation Memory Format, and each can be consumed independently by a tool that only needs one language pair, or together by a tool that reads `manifest.xml` to reconstruct the full set. `en-es.xlf` and `en-fr.xlf` share a common source language and MAY be aligned by `<unit>` and `<segment>` `id` as described in Package Structure; `fr-de.xlf` has no source language in common with the other two files and is not part of that alignment.

As described in Package Structure, the same three files MAY instead be organized into one folder per language pair:

```
tm-package.zip
    manifest.xml
    en-es/
        en-es.xlf
    en-fr/
        en-fr.xlf
    fr-de/
        fr-de.xlf
```

with the manifest updated to match:

```
<tmPackage xmlns="urn:oasis:names:tc:xliff:tm-manifest:1.0" version="1.0">
  <file href="en-es/en-es.xlf" srcLang="en" trgLang="es"/>
  <file href="en-fr/en-fr.xlf" srcLang="en" trgLang="fr"/>
  <file href="fr-de/fr-de.xlf" srcLang="fr" trgLang="de"/>
</tmPackage>
```

Both layouts are valid and equivalent: a conformant consumer resolves every file through the `href` value recorded in the manifest rather than by assuming either layout.

7 Creating and Searching a Translation Memory Store

This section is informative and does not define conformance requirements. It describes practical, non-mandatory approaches for creating and querying a translation memory store as described in XLIFF as a Translation Memory Format. Implementers MAY use either of these approaches, a combination of them, or an approach not described here; this document does not mandate a particular storage or retrieval technology.

The packaging convention described in Exchange of Multilingual Translation Memory Data is intended for exchanging translation memory data between parties and tools, not as an implementation architecture. An implementation is not expected to query a ZIP package directly: it typically loads the bilingual XLIFF files it receives, or the ones it produces by harvesting as described in Harvesting Translation Memory Entries, into whichever storage and retrieval mechanism it uses internally, of which the following are two examples.

Fuzzy Search Versus Semantic Search

The two approaches described in this section answer different questions. Fuzzy search, as provided by an XML-aware database (through XPath/XQuery string functions or a similarity extension) or by a SQL database (through trigram indexes, edit-distance functions, or full-text search), measures how similar two strings are as sequences of characters. It reliably finds near-duplicates that differ by a word, a typo, or a small edit, but has no notion of meaning: it can rank a sentence whose negation flips its meaning as a close match to the original, and it will not find a paraphrase that shares no wording at all.

Semantic search, as provided by a vector database over text embeddings, measures how similar two passages are in meaning, independent of wording. It finds paraphrases that a fuzzy search misses, but is not guaranteed to penalize a small, meaning-changing edit as heavily as a fuzzy search would, since embeddings can place semantically related but contradictory statements close together in vector space. This complementary blind spot is why Vector Database and Similarity Search recommends blending a semantic score with a fuzzy score rather than relying on either alone.

Organizations with an existing SQL-based translation memory engine, many of which already parse a source/target pair out of a TMX `<tu>`/`<tuv>` and store it in relational tables, do not need to replace that engine to work with a store described in this document: the same import step can instead read `<source>`/`<target>` content directly out of XLIFF `<segment>` elements. This is a low-effort migration path that reuses existing search and scoring logic, but on its own it does not bring the context-aware retrieval described in XML-Aware Database and XPath Retrieval or the paraphrase-level retrieval described in Vector Database and Similarity Search: a relational schema built around a flat source/target pair typically discards the same document order, `<file>` origin, and grouping that a flat TMX export discards, unless the importer is extended to capture them as additional columns.

7.1 XML-Aware Database and XPath Retrieval

Because a translation memory store described in this document is plain, well-formed XLIFF, it can be loaded directly into an XML-aware database and queried with XPath or XQuery, without any transformation of its content. A query can match on `<source>` text, restrict candidates to a particular `<file>` `original` value or Provenance attribute as described in Provenance Metadata for Translation Memory Entries, or exploit the preserved document order described in Structural Order and Segment Context to retrieve a match together with its neighboring segments.

Examples of XML-aware databases that support this style of storage and query include, without limitation, BaseX, eXist-db, MarkLogic, and Oracle XML DB. Naming these products is illustrative only and does not constitute an endorsement or a requirement to use any of them.

For example, the following excerpt of a translation memory store, taken from the store shown in Example, contains two entries with identical source text, harvested from two different documents:

```
<file id="f1" original="manual/chapter3.html">
  <unit id="tm-0002">
    <segment>
      <source>Download your certificate before the end of the fiscal year.</source>
      <target>Descargue su certificado antes de que finalice el año fiscal.</target>
    </segment>
  </unit>
</file>
<file id="f2" original="app/ui-strings.json">
  <unit id="tm-1001">
    <segment>
      <source>Download your certificate before the end of the fiscal year.</source>
      <target>Descargue su certificado antes de que finalice el año fiscal.</target>
    </segment>
  </unit>
</file>
```

An XPath query that matches on source text alone:

```
//unit[segment/source = 'Download your certificate before the end of the fiscal year.']
```

returns both `tm-0002` and `tm-1001`, since both segments share the same exact source text. If the unit currently being translated originated from `manual/chapter3.html`, the query can be narrowed to prefer an in-context exact match, as described in Structural Order and Segment Context, by also constraining the enclosing `<file>` element's `original` attribute:

```
//file[@original = 'manual/chapter3.html']
    /unit[segment/source = 'Download your certificate before the end of the fiscal year
```

which returns only `tm-0002`, the entry harvested from the same document as the unit being translated.

7.2 Vector Database and Similarity Search

Alternatively, the `<source>` (and, where useful, `<target>`) text of each harvested `<segment>` can be converted to a numeric embedding by a text embedding model and indexed in a vector database, with each vector referencing the `<unit>` and `<file>` it was derived from. Retrieval is then performed as a similarity search over embeddings rather than an exact or XPath-based text match, which can surface fuzzy matches that share meaning but not exact wording. As with the XML-database approach, the enclosing `<file>` `original` attribute and Provenance metadata remain available as filters that can be applied alongside, or after, the similarity search to narrow candidates.

Examples of vector databases that support this style of retrieval include, without limitation, LanceDB, Milvus, Weaviate, Qdrant, and pgvector, a vector extension for PostgreSQL. Naming these products is illustrative only and does not constitute an endorsement or a requirement to use any of them.

Continuing the example from XML-Aware Database and XPath Retrieval, the same two entries could instead be indexed as vectors, each referencing the `<unit>` and originating `<file>` it was derived from (embedding values abbreviated for illustration):

```
tm-0002  original=manual/chapter3.html  "Download your certificate before the end of th
tm-1001  original=app/ui-strings.json   "Download your certificate before the end of th
```

Unlike an XPath query, a similarity search is not limited to exact text. A query using a paraphrase of the same request, which shares no exact wording with either stored segment:

```
"Please download your tax certificate before the fiscal year ends."
```

is converted to an embedding and compared against the indexed vectors, returning both `tm-0002` and `tm-1001` ranked by similarity score (for example, 0.93 for each), even though an XPath query for this exact string, as in XML-Aware Database and XPath Retrieval, would return no results at all. As in that example, restricting the search to vectors whose `original` metadata equals `manual/chapter3.html` narrows the result to `tm-0002` alone.

In practice, semantic similarity is often combined with a string-similarity ("fuzzy") metric into a single blended score, rather than used alone. Embeddings alone can rank a candidate that changes the meaning nearly as highly as one that only changes the wording; a fuzzy string comparison of the same candidates penalizes that difference, while still tolerating the minor spelling or punctuation variation that an exact-text query would reject outright. Blending the two, for example by averaging a semantic similarity score with a string-similarity score such as longest-common-subsequence, gives a single ranking that neither method reliably produces on its own.

Beyond translation-pair lookup, a concordance-style search, returning every stored entry whose `<source>` or `<target>` contains a given text fragment across every language present in the store, is a common complementary query mode. Either approach in this section can support it: an XPath `contains()` predicate for an XML-aware database, or a plain substring or full-text index alongside the vector index for a vector database. It is particularly useful for terminology review and quality checks that are not aimed at finding a reusable translation.

8 Safety, Security, and Data Protection Considerations

Translation memory content frequently originates from source documents that were never intended for public redistribution, and may contain personal data or confidential business information carried over from the original text. Storing that content as plain XLIFF, rather than as an opaque TM database, does not change its sensitivity; implementers SHOULD apply the same access controls, storage protections, and redistribution policies to XLIFF-based TM files and packages as they would to any other document containing the same source content.

The Provenance module attributes described in Provenance Metadata for Translation Memory Entries, in particular `person`, can themselves constitute personal data about the translators and reviewers who worked on the content. Implementers SHOULD consider applicable data protection requirements (such as consent and retention limits) before populating, storing, or exchanging `person` values in translation memory data, and MAY omit `person` in favor of `agent`, `organization`, or `tool` where individual identification is not required.

Translation memory exchange packages, as described in Exchange of Multilingual Translation Memory Data, are ordinary ZIP archives. Implementations that unpack them SHOULD apply the same precautions recommended for processing any untrusted ZIP archive, including validating that archive entries do not escape the intended extraction directory (path traversal) and guarding against archives crafted to consume excessive disk space or memory when decompressed.

9 Conformance

9.1 Translation Memory Store

An XLIFF document **conforms** to this document as a Translation Memory Store if:

1. It is a conformant [XLIFF-2.3-part2] (or later) document.
2. Every stored translation memory entry is represented as an XLIFF `<unit>` containing one `<segment>` per entry, with `<source>` and, where a translation is available, `<target>` content, and no lossy conversion of inline content has been applied to represent that entry.
3. Where provenance information is recorded for an entry, it is expressed using the XLIFF 2.3 Provenance module rather than free-text notes or properties.

9.2 Translation Memory Exchange Package

A ZIP archive **conforms** to this document as a Translation Memory Exchange Package if:

1. It contains a `manifest.xml` file at its root, as described in Manifest File.
2. Every XLIFF file listed in the manifest is present in the archive and conforms to this document as a Translation Memory Store.
3. Every XLIFF file listed in the manifest declares a single `srcLang` and a single `trgLang`, matching the language pair recorded for that file in the manifest.

Appendix A License, Document Status and Notices (Normative)

A.1 Document Status

This document was last revised or approved by the OASIS XLIFF TC on the above date. The level of approval is also listed above. Check the "Latest version" location noted above for possible later revisions of this document. Any other numbered Versions and other technical work produced by the Technical Committee (TC) are listed at https://groups.oasis-open.org/communities/tc-community-home2?CommunityKey=3d0f1f56-8477-4b53-9b14-018dc7d3eecf#technical.

TC members should send comments on this document to the TC's email list. Others should send comments to the TC's public comment list, after subscribing to it by following the comment instructions at the TC's comments list web page at https://www.oasis-open.org/committees/comments/index.php?wg_abbrev=xliff.

A.2 License and Notices

Appendix B References

This section contains the normative and informative references that are used in this document.

Informative references are either specific or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the reference document (including any amendments) applies. While any hyperlinks included in this section were valid at the time of publication, OASIS cannot guarantee their long term validity.

B.1 Normative References

[**XLIFF-2.3-part2**] XLIFF Version 2.3 Part 2: Extended. Edited by Rodolfo M. Raya and Lucía Morado Vázquez. 8 July 2026. OASIS Committee Specification Draft 01. *https://docs.oasis-open.org/xliff/xliff-core/v2.3/xliff-v2.3-part2-extended.html*.

[**RFC2119**] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. March 1997. *https://www.rfc-editor.org/rfc/rfc2119*.

[**ZIP APPNOTE**] PKWARE, Inc. *.ZIP File Format Specification*. *https://pkware.cachefly.net/webdocs/case-studies/APPNOTE.TXT*.

B.2 Informative References

[**TMX 1.4b**] Translation Memory eXchange (TMX) Version 1.4b. Edited by Yves Savourel. OSCAR Recommendation, 26 April 2005. *https://web.archive.org/web/20060528180452/http://www.lisa.org/standards/tmx/tmx.html*.

[**XLIFF-2.3-provenance**] XLIFF Version 2.3 Provenance Module (Work in Progress). OASIS XLIFF Technical Committee. Not yet published; referenced here for context while under development in the same working repository as this document.

Appendix C Acknowledgments (Informative)

C.1 Leadership

The following individuals have had significant leadership positions during the development of this document, not just this version of the document, and they are gratefully acknowledged:

- Chairs
  - Dr. Lucía Morado Vázquez, University of Geneva
  - Yoshito Umaoka, Individual
- Secretaries
  - Rodolfo M. Raya, Maxprograms
- Editors
  - Rodolfo M. Raya, Maxprograms

C.2 Special Thanks

The following individuals have made substantial contributions to this document, not just this version of the document, and their contributions are gratefully acknowledged:

- Mihai Niță, Google LLC
- Mathijs Sonnemans, Individual

C.3 Participants

The following individuals were members of this committee during the creation of this document, not just this version of the document, and their contributions are gratefully acknowledged:

< Placeholder: full list of individuals who were members of the OASIS XLIFF TC during the creation of this document. >

Appendix D Changes From Previous Version (Informative)

None. This is the first published draft of this document.

D.1 Revision History

None.